Securing Agentic Browsers Against Malicious Web Interactions.

Challenge Overview

AI-powered agents that can autonomously control web browsers are increasingly used for automated workflows such as data extraction, form filling, ticket booking, and online task automation. These *Agentic Browsers* rely on Large Language Models (LLMs) to interpret web content and decide actions such as clicking buttons, entering text, and navigating pages.

However, web environments are inherently adversarial. Malicious web pages can embed deceptive instructions, hidden prompts, misleading UI elements, or dynamically injected content that can manipulate the agent into performing unsafe or unintended actions. Traditional browser security mechanisms are designed for humans and are insufficient for AI-driven agents.

This challenge focuses on designing a security framework that protects agentic browsers from malicious web interactions while preserving task completion efficiency.

Objectives

Malicious Web Content Detection

- Detect prompt injection attempts embedded in visible or hidden web content.
- Identify deceptive UI elements such as fake buttons, misleading forms, and clickjacking.
- Analyze dynamically generated DOM content for adversarial behavior.

Secure Action Mediation

- Intercept and validate agent actions before execution.
- Block, modify, or require confirmation for high-risk browser actions.
- Enforce safety policies without significantly degrading usability.

Explainable Risk Assessment

- Generate a risk score for each web interaction.
- Provide human-readable explanations for blocked or flagged actions.

Evaluation Criteria

Attack Detection Accuracy

- Detection rate of prompt injection and malicious web attacks.
- Precision, recall, and F1-score for attack classification.

Task Success Rate

- Ability of the agent to successfully complete legitimate tasks.
- Minimal disruption to normal browsing workflows.

False Positive and False Negative Rates

- Incorrectly blocked benign actions.
- Missed detection of adversarial content.

Performance and Latency

- Overhead introduced per browser action.
- Real-time responsiveness of the security layer.

Interpretability

- Clarity of security decisions and risk explanations.
- Transparency of detected threats and mitigation actions.

Implementation Guidelines and Suggestions

General Implementation Rules

- Core security mechanisms must be implemented from scratch.
- Copying complete solutions from GitHub, Kaggle, or similar platforms is prohibited.
- Teams must clearly document system architecture, models used, and decision logic.

Browser Environment

- Teams may use Playwright, Puppeteer, or Selenium for browser automation.
- A headless or visible Chromium-based browser may be used.
- The agent should be capable of reading page content and executing browser actions.

भारतीय प्रौद्योगिकी संस्थान कानपुर
Indian Institute of Technology Kanpur

Attack Scenarios

Participants should consider defending against:

- Prompt injection via visible web text (e.g., "Ignore previous instructions").
- Hidden instructions using CSS (e.g., `display:none`, tiny fonts).
- Deceptive buttons and forms designed to exfiltrate sensitive information.
- JavaScript-based dynamic content injection after page load.
- Phishing-style login pages and fake confirmation dialogs.

Advanced Models and Techniques

0.0.1 DOM and Content Analysis

- Parse and analyze DOM trees for suspicious structures.
- Detect hidden or low-visibility text nodes.
- Classify content using NLP-based threat detection.

0.0.2 Policy-Based Action Control

- Implement allowlists and blocklists for actions.
- Use rule-based and ML-based risk scoring mechanisms.
- Introduce multi-step verification for sensitive actions.

0.0.3 LLM-Assisted Threat Reasoning

- Use LLMs to reason about intent behind web content.
- Compare extracted instructions against agent goals.

0.0.4 Scalability Considerations

- Ensure efficient handling of large and complex web pages.
- Optimize DOM parsing and action interception pipelines.

Deliverables

Working Prototype

- Secure agentic browser implementation.
- Demonstration of defense against at least three attack types.
- Risk scoring and explainability output.

Source Code

- Complete, modular, and well-documented codebase.
- Repository link (GitHub/GitLab) with clear setup instructions.

भारतीय प्रौद्योगिकी संस्थान कानपुर
Indian Institute of Technology Kanpur

Documentation

- Architecture description and threat model.
- Explanation of detection logic and policies.
- Performance and evaluation results.

Demo Video

- 5–10 minute video showcasing attacks and defenses.
- Explanation of system behavior and decisions.

Submission Format

- **Platform:** Submissions must be made using official college email IDs.
- **File Structure:**
  - Source code directory
  - Documentation files (`README.md`, `TechnicalDocumentation.pdf`)
  - Demo video link (YouTube, Vimeo, or Google Drive)

I3iHub
भारतीय प्रौद्योगिकी संस्थान कानपुर
Indian Institute of Technology Kanpur